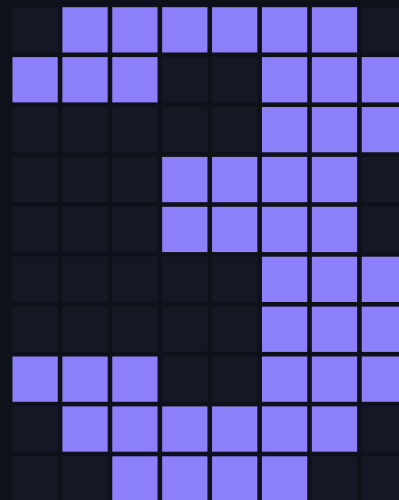




Build a neural network that reads handwriting

One Python file. Real MNIST digits.
A learning loop you can watch.

*You will write every line that matters -
the layers, the loss, the update.*



what number is this?

Guess. Measure. Change. Repeat.

A network starts with random weights, so its first guesses are bad - and that is exactly where learning begins.

Each round measures how wrong the guess was, finds which weights caused the error, and nudges every weight a little.



Every line of code today belongs to one of these five moves.

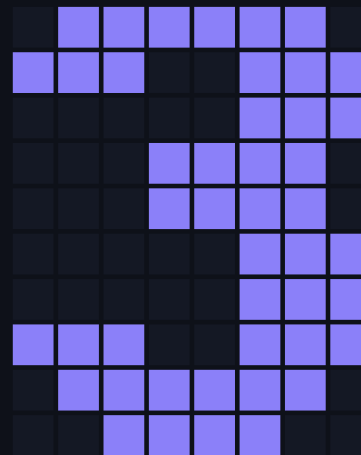


A picture is 784 numbers

A 28 x 28 grayscale image: one brightness value per pixel, 0 to 255.

Flatten the square into one row of 784 numbers - no pixel is lost.

Divide by 255 so every value lands between 0 and 1.



```
x = image.view(-1, 784) / 255.0
```

Weighted sums, plus a bias

A layer multiplies each input by an adjustable weight, adds the products, then adds a bias - producing a new set of numbers.

The key word is "adjustable": learning is nothing more than changing these weights.

$(\text{images}, 784) @ (784, 128) + (128) \rightarrow (\text{images}, 128)$



The inner dimensions match; the outer ones survive. @ runs every multiply-and-add at once.



Build one learned layer

```
class Linear:
    def __init__(self, in_features, out_features):
        std = (2 / in_features) ** 0.5
        self.weights = torch.randn(in_features, out_features) * std
        self.bias = torch.zeros(out_features)

        self.weights.requires_grad_()
        self.bias.requires_grad_()

    def forward(self, x):
        return x @ self.weights + self.bias

    def parameters(self):
        return [self.weights, self.bias]
```

READ IT AS

Random weights,
zero biases.

Ask PyTorch to track them.
forward() makes the guess.
parameters() = every value
the loop may change.

✓ Checkpoint 1 · three images through one layer. Expect (3, 784) -> (3, 128).



Add a bend

Stack linear layers with nothing between them and the whole stack is still just one linear layer.

Digits are messy; the model needs bends to draw richer boundaries.

```
class ReLU:
    def forward(self, x):
        return x.clamp(min=0)
```

READ IT AS

Below zero becomes zero.

Everything else passes through.

$[-2, 0, 3] \rightarrow [0, 0, 3]$



784 -> 128 -> 128 -> 10

```
class NeuralNetwork:
    def __init__(self, input_size, hidden_size, output_size):
        self.layer1 = Linear(input_size, hidden_size)
        self.layer2 = Linear(hidden_size, hidden_size)
        self.layer3 = Linear(hidden_size, output_size)
        self.relu1 = ReLU()
        self.relu2 = ReLU()
```

TYPE ALL OF IT

All three methods, or Checkpoint 2 fails.
parameters() adds the three lists into
one flat list of six tensors:
three weights, three biases.

```
def forward(self, x):
    x = self.layer1.forward(x)
    x = self.relu1.forward(x)
    x = self.layer2.forward(x)
    x = self.relu2.forward(x)
    x = self.layer3.forward(x)
    return x

def parameters(self):
    return (
        self.layer1.parameters()
        + self.layer2.parameters()
        + self.layer3.parameters()
    )
```

✓ Checkpoint 2 · four images become ten scores each. Expect (4, 784) -> (4, 10).

Ten scores, one guess

01

argmax

largest score = the guess

02

logit

the name for one raw score

03

softmax

scores into probabilities

Same guess with or without softmax - biggest score wins. We never need it to train.



Cross-entropy compares all ten scores with the correct digit - that becomes the loss.



Load MNIST

```
def load_data():
    train = torchvision.datasets.MNIST(
        root="./data", train=True, download=True)
    test = torchvision.datasets.MNIST(
        root="./data", train=False, download=True)

    x = train.data.float().view(-1, 784) / 255.0
    target = train.targets
    x_test = test.data.float().view(-1, 784) / 255.0
    target_test = test.targets
    return x, target, x_test, target_test

def accuracy(scores, targets):
    return (scores.argmax(dim=1) == targets).float().mean().item()
```

READ IT AS

Learn on one set,
test on a separate set.
Flatten to 784,
scale 0-255 to 0-1,
keep each digit as the target.
accuracy: how often the biggest
score names the right digit.



One number, then every gradient

`loss = cross_entropy(prediction, target)` is high when the right digit scores poorly, low when it wins - the single signal learning reduces.

`loss.backward()` walks the recorded operations in reverse and reports how each weight contributed to the error.

```
loss <- scores <- layers <- every weight and bias
```



A gradient is a direction: changing this value that way would raise the loss.



One small step, then clear

Each gradient points
toward more error,
so subtract a little
of it from each weight.

```
loss.backward()

with torch.no_grad():
    for parameter in model.parameters():
        parameter -= learning_rate * parameter.grad

for parameter in model.parameters():
    parameter.grad.zero_()
```

READ IT AS

Step opposite the gradient, a little.
no_grad(): this is the change itself.
Clear the grads, or the next round
mixes old directions with new.

✓ Checkpoint 3 - one manual step makes a fixed problem's loss smaller.



Nothing is hidden

```
def main():
    x, target, x_test, target_test = load_data()
    model = NeuralNetwork(784, 128, 10)
    learning_rate = 0.1
    epochs = 10

    for epoch in range(1, epochs + 1):
        prediction = model.forward(x)
        loss = torch.nn.functional.cross_entropy(prediction, target)
        loss.backward()

        with torch.no_grad():
            for parameter in model.parameters():
                parameter -= learning_rate * parameter.grad

        for parameter in model.parameters():
            parameter.grad.zero_()

    if epoch == 1 or epoch % 5 == 0 or epoch == epochs:
        report(epoch, loss, model, x_test, target_test)
```

Ten rounds, then sixty

~2.48

starting loss

~1.69

loss after 10 epochs

~65%

test accuracy at 10 epochs



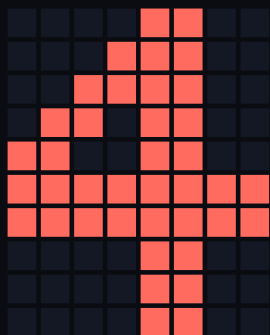
Change epochs 10 -> 60: loss ~0.48, test accuracy ~88%. Same network, more rounds.

Blind guessing is 10%. Read the trend, not one exact number: loss down, accuracy up.

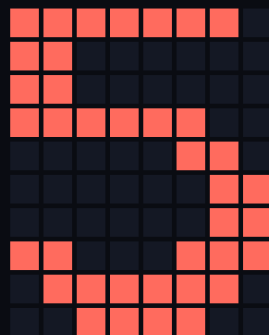
Illustrative seeded output - facilitator, use the numbers you measured on the event machine.



It matches pixels - it does not know digits



guess 9 · truth 4



guess 3 · truth 5

It learned pixel patterns that usually work. Draw a 4 like a 9 and it has nothing else to fall back on.



Keep the file. Change one thing.

- 1 Download mnist_network.py
- 2 Pick one experiment
- 3 Predict what will happen
- 4 Change one value
- 5 Run and compare

TRY ONE

epochs 10 -> 60
learning rate 0.1 -> 0.01
hidden size 128 -> 32
break the init (std = 1.0)
layer1_weight_grid(model)

You built the layers, the gradients, and the update. The high-level PyTorch tools package the same machinery.